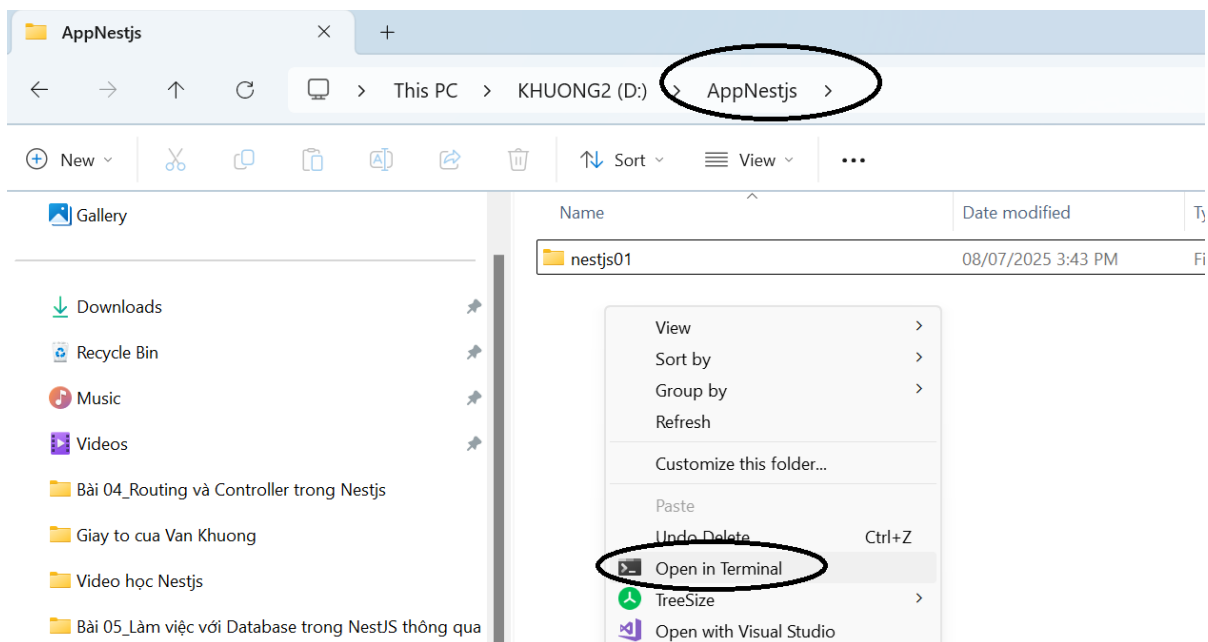


## BÀI 05\_02: LÀM VIỆC VỚI DATABASE MONGODB TRONG NESTJS THÔNG QUA TYPEORM

**Mục tiêu bài 05:** Bài này sẽ giới thiệu đến các bạn các vấn đề về TypeORM, cài đặt và cấu hình kết nối dữ liệu vào Nestjs thông qua TypeORM. Giới thiệu các bạn về các hàm cơ bản thao tác xử lý dữ liệu MongoDB trong Nestjs thông qua TypeORM.

### 5.1. TẠO DỰ ÁN VÀ CÀI ĐẶT TYPEORM CHO DỰ ÁN NESTJS

- Đầu tiên các bạn tạo dự án **Bai05\_02** trong thư mục **AppNestjs** đã tạo ở bài trước, bằng cách: click chuột phải vào vùng trống của thư mục đó, chọn **Open in Terminal**



- Xuất hiện giao diện **cmd**, các bạn gõ lệnh: **nest new Bai05\_02** (để tạo dự án Bai05\_02). Chọn **npm**, tiếp tục bấm **Enter**. Chờ cho quá trình tạo dự án diễn ra và báo quá trình tạo dự án thành công.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWin
dows

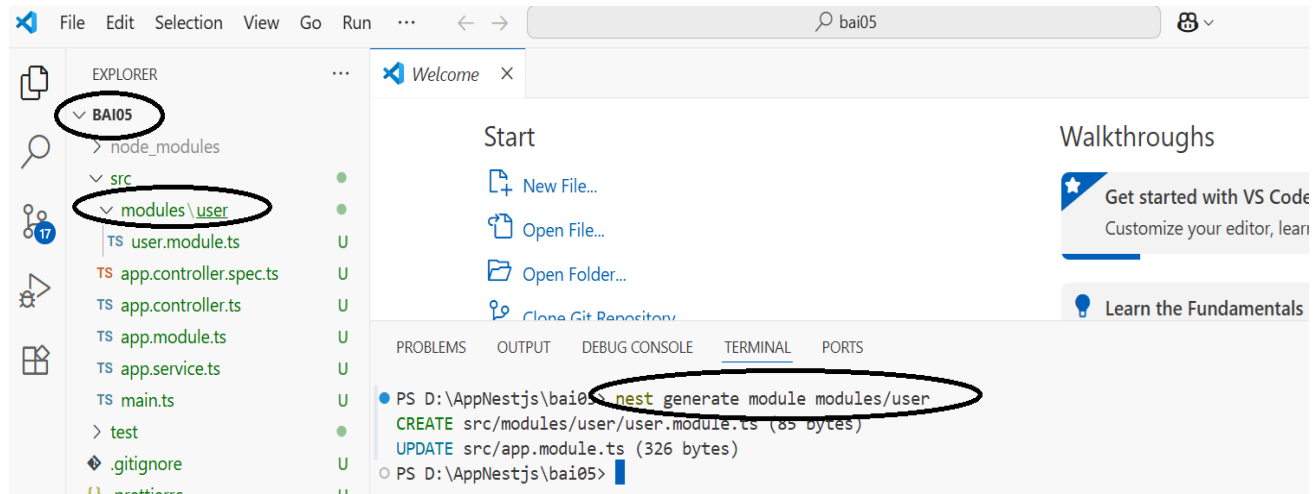
PS D:\AppNestjs> nest new Bai05_02
✦ We will scaffold your app in a few seconds..

✔ Which package manager would you ❤️ to use? npm
CREATE bai05_02/.prettierrc (54 bytes)
CREATE bai05_02/eslint.config.mjs (869 bytes)
CREATE bai05_02/nest-cli.json (179 bytes)
CREATE bai05_02/package.json (2050 bytes)
CREATE bai05_02/README.md (5126 bytes)
CREATE bai05_02/tsconfig.build.json (101 bytes)
CREATE bai05_02/tsconfig.json (702 bytes)
CREATE bai05_02/src/app.controller.ts (286 bytes)
CREATE bai05_02/src/app.module.ts (259 bytes)
CREATE bai05_02/src/app.service.ts (150 bytes)
CREATE bai05_02/src/main.ts (236 bytes)
CREATE bai05_02/src/app.controller.spec.ts (639 bytes)
CREATE bai05_02/test/jest-e2e.json (192 bytes)
CREATE bai05_02/test/app.e2e-spec.ts (699 bytes)

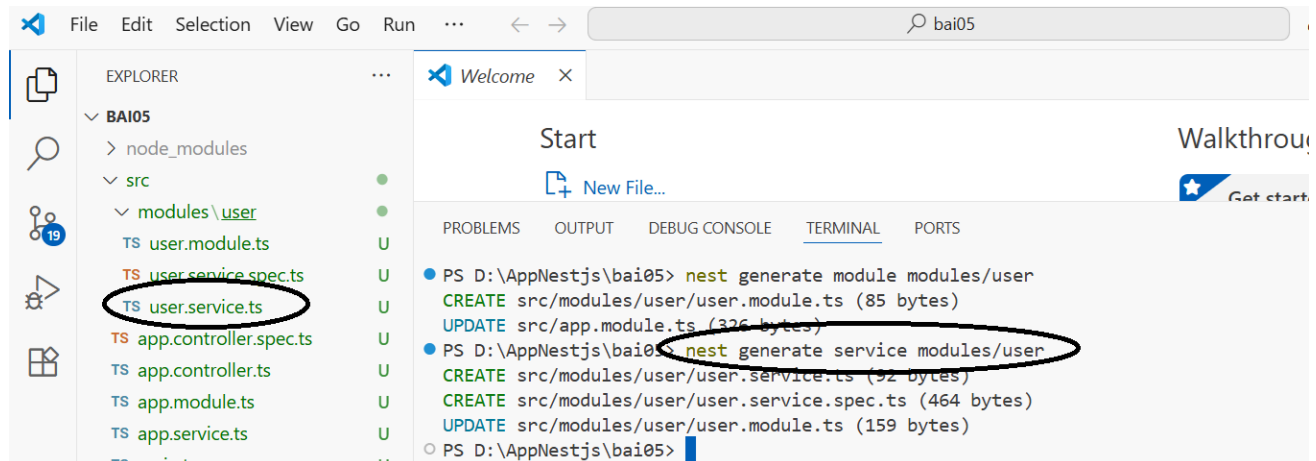
>>>> Installation in progress...
```

- Mở chương trình **VSCode** lên, mở thư mục dự án **Bai05\_02**

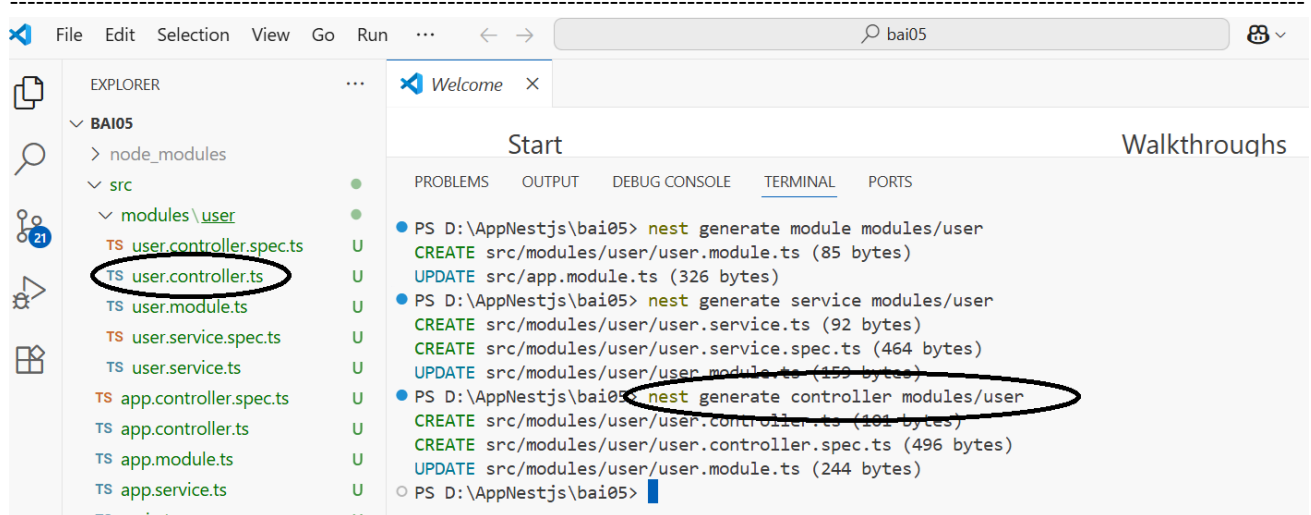
- Các bạn mở **Terminal** của VScode gõ dòng lệnh: **nest generate module modules/user**.  
Đợi một lát sẽ thấy một **module user** tạo ra trong dự án (nằm trong thư mục **modules**).



- Ở Terminal các bạn gõ dòng lệnh: **nest generate service modules/user** để tạo **service**.  
Đợi 1 lát sẽ tạo ra tập tin **user.service.ts**



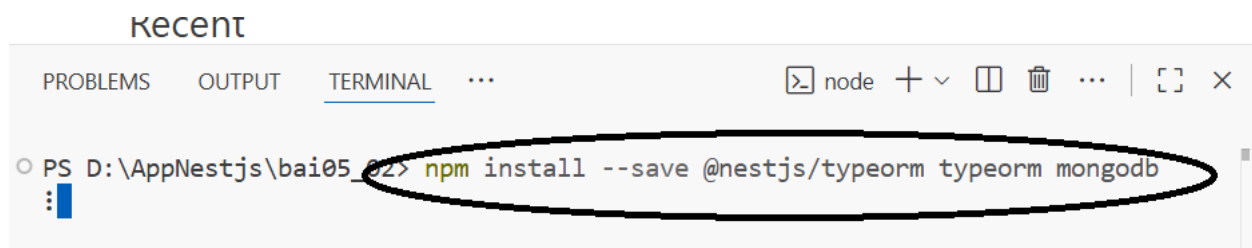
- Tiếp tục ở Terminal các bạn gõ dòng lệnh: **nest generate controller modules/user** để tạo **controller**. Đợi 1 lát sẽ tạo ra tập tin **user.controller.ts**



- Tiếp tục, các bạn Cài đặt **TypeORM** vào dự án **Bai05** bằng cách tại Terminal gõ dòng lệnh: **npm install --save @nestjs/typeorm typeorm mongodb**

*Giải thích lệnh:*

- **@nestjs/typeorm**: module typeorm cho Nestjs
- **typeorm**: thư viện typeorm
- **mongodb**: Driver cho MongoDB



- Như vậy các bạn đã tạo dự án **Nestjs** và cài đặt **TypeORM** vào dự án.

## 5.2. CẤU HÌNH KẾT NỐI TYPEORM KẾT NỐI CƠ SỞ DỮ LIỆU MONGODB

- Các bạn mở file **app.module.ts** và viết code như bên dưới:



- Chương trình sử dụng cơ sở dữ liệu **MongoDB** là **nest\_typeorm\_demo**, nên các bạn cần tạo một cơ sở dữ liệu **MongoDB** tên là: **MongoDB**

**Create Database**

Database Name: nest\_typeorm\_demo

Collection Name: users

☐ Time-Series  
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

> Additional preferences (e.g. Custom collation, Clustered collections)

Cancel Create Database

### 5.3. ĐỊNH NGHĨA ENTITY

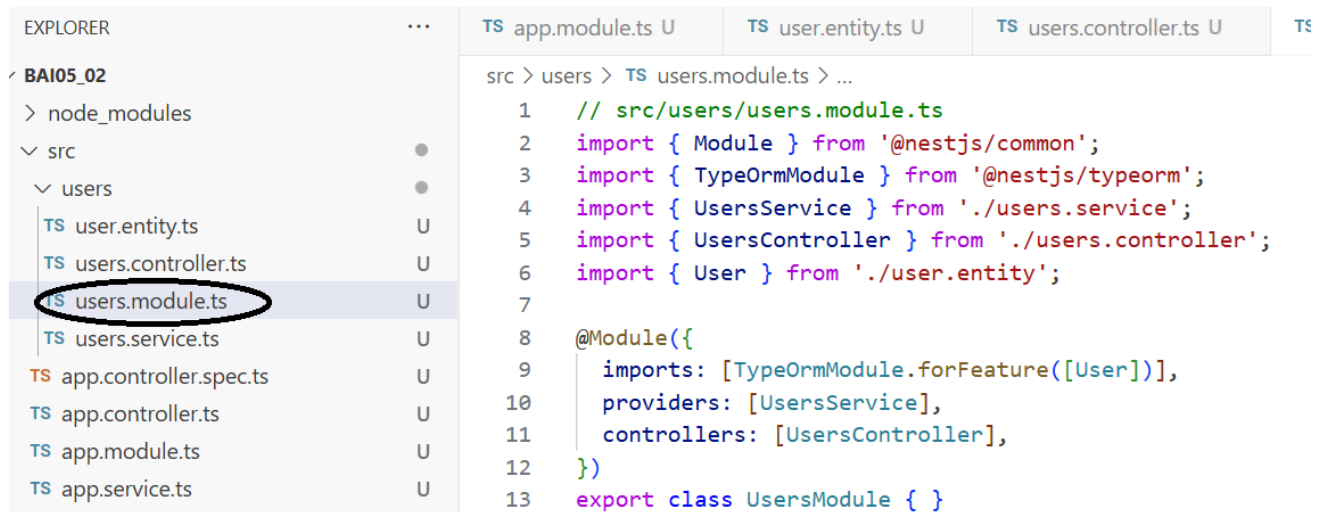
- Để tạo Entity cho ứng dụng đang làm, các bạn tiếp tục tạo trong thư mục **user** tập tin **user.entity.ts** và viết code như sau:

```

1  // src/users/user.entity.ts
2  import { Entity, ObjectIdColumn, Column } from 'typeorm';
3  import { ObjectId } from 'mongodb';
4
5  @Entity('users')           // tên collection: users
6  export class User {
7      @ObjectIdColumn()
8      _id: ObjectId;
9
10     @Column()
11     name: string;
12
13     @Column()
14     email: string;
15
16     @Column()
17     age?: number;
18 }
    
```

## 5.5. IMPORT ENTITY VÀO MODULE

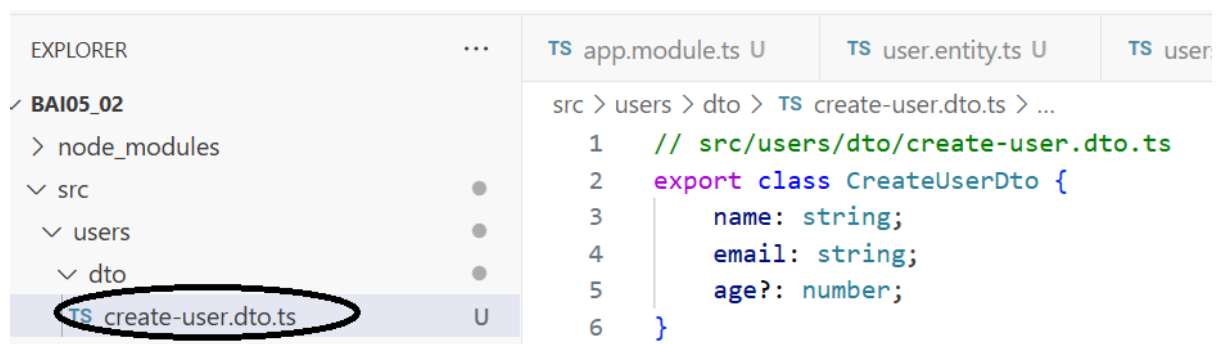
- Để Import Entity cho ứng dụng đang làm, các bạn tiếp tục mở tập tin **user.module.ts** và viết code như sau:



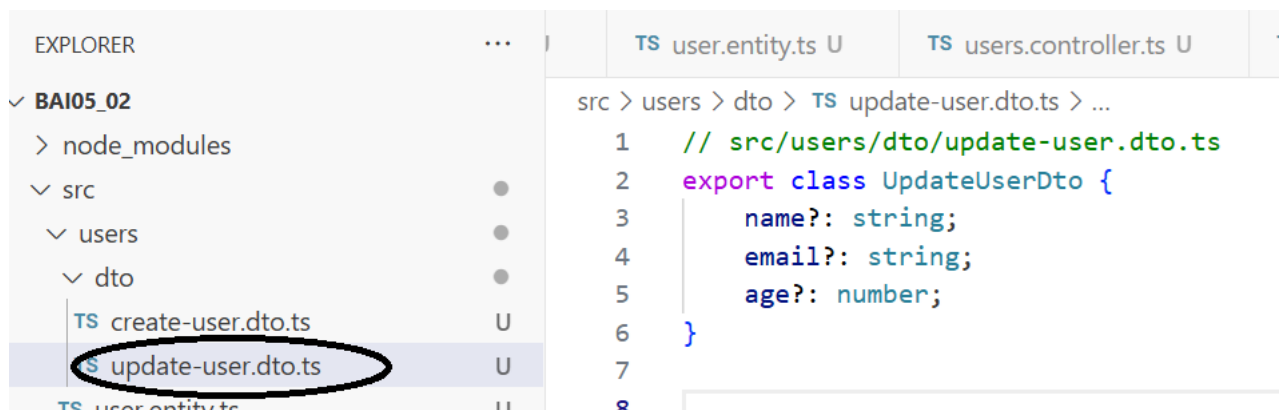
## 5.5. TẠO CÁC DTO

**DTO** là viết tắt của Data Transfer Object (Đối tượng Truyền dữ liệu). Nó là một **class** hoặc **interface** được dùng để mô tả cấu trúc dữ liệu mà **client** sẽ gửi lên hoặc **server** trả về.

- Các bạn tạo thư mục **dto** trong thư mục **users**. Tạo tập tin **create-user.dto.ts** trong thư mục **dto** và viết **code** như bên dưới:



- Tiếp tục tạo tập tin **update-user.dto.ts** trong thư mục **dto** và viết code như bên dưới:



## 5.6. REPOSITORY TRONG TYPEORM

**Repository** là nơi thực hiện các thao tác với Database như **CRUD** (Create, Read, Update, Delete).

- Bây giờ các bạn thực hiện **Inject Repository** vào **Service** và tạo các hàm cơ bản xử lý cơ sở dữ liệu bằng cách: mở tập tin **user.service.ts** và viết code như bên dưới:

src > users > TS users.service.ts > ...

```

1  // src/users/users.service.ts
2  import { Injectable, NotFoundException } from '@nestjs/common';
3  import { InjectRepository } from '@nestjs/typeorm';
4  import { MongoRepository } from 'typeorm';
5  import { User } from '../user.entity';
6  import { ObjectId } from 'mongodb';
7  import { CreateUserDto } from '../dto/create-user.dto';
8  import { UpdateUserDto } from '../dto/update-user.dto';
9
10 @Injectable()
11 export class UsersService {
12   constructor(
13     @InjectRepository(User)
14     private readonly usersRepository: MongoRepository<User>,
15   ) { }
16
17   private toDto(user: User | null) {
18     if (!user) return null;
19     const id = (user as any)._id?.toHexString ? (user as any)._id.toHexString() : user._id;
20     return { id, name: user.name, email: user.email, age: user.age };
21   }
22
23   async create(dto: CreateUserDto) {
24     const res = await this.usersRepository.insertOne(dto);
25     const saved = await this.usersRepository.findOneBy({ _id: res.insertedId });
26     return this.toDto(saved);
27   }
28
29   async findAll() {
30     const users = await this.usersRepository.find();
31     return users.map(u => this.toDto(u));
32   }
33
34   async findOne(id: string) {
35     const user = await this.usersRepository.findOneBy({ _id: new ObjectId(id) });
36     if (!user) throw new NotFoundException('User not found');
37     return this.toDto(user);
38   }
39
40   async update(id: string, dto: UpdateUserDto) {
41     const user = await this.usersRepository.findOneBy({ _id: new ObjectId(id) });
42     if (!user) throw new NotFoundException('User not found');
43     Object.assign(user, dto);
44     const saved = await this.usersRepository.save(user);
45     return this.toDto(saved);
46   }
47
48   async remove(id: string) {
49     const r = await this.usersRepository.deleteOne({ _id: new ObjectId(id) });
50     if (r.deletedCount === 0) throw new NotFoundException('User not found');
51     return { deleted: true };
52   }
53 }

```

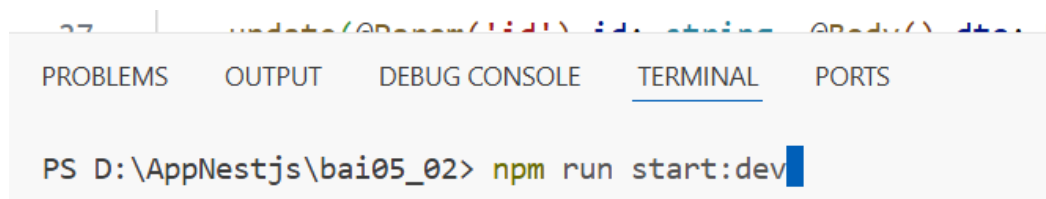
## 5.7. CONTROLLER GỌI SERVICE ĐỂ THỰC HIỆN CRUD

- Để gọi **Service** để thực hiện 2 hàm (phương thức) với cơ sở dữ liệu đã tạo các bạn mở tập tin **user.controller.ts** và viết code như bên dưới:

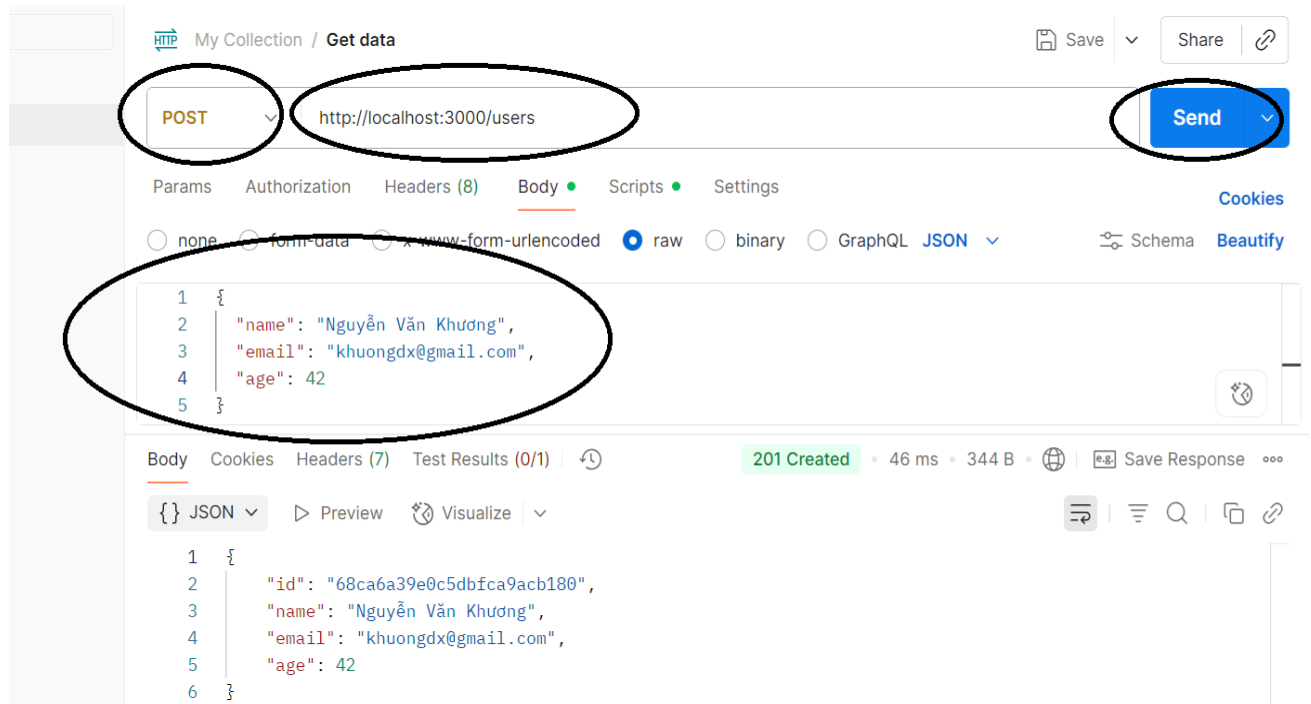
```
src > users > TS users.controller.ts > UsersController > update
1  // src/users/users.controller.ts
2  import { Controller, Get, Post, Body, Param, Put, Delete } from '@nestjs/common';
3  import { UsersService } from '../users.service';
4  import { CreateUserDto } from '../dto/create-user.dto';
5  import { UpdateUserDto } from '../dto/update-user.dto';
6
7  @Controller('users')
8  export class UsersController {
9      constructor(private readonly usersService: UsersService) { }
10
11      @Post()
12      create(@Body() dto: CreateUserDto) {
13          return this.usersService.create(dto);
14      }
15
16      @Get()
17      findAll() {
18          return this.usersService.findAll();
19      }
20
21      @Get(':id')
22      findOne(@Param('id') id: string) {
23          return this.usersService.findOne(id);
24      }
25
26      @Put(':id')
27      update(@Param('id') id: string, @Body() dto: UpdateUserDto) {
28          return this.usersService.update(id, dto);
29      }
30
31      @Delete(':id')
32      remove(@Param('id') id: string) {
33          return this.usersService.remove(id);
34      }
35  }
```

- Như vậy các hàm xử lý dữ liệu đã được đưa vào **Controller**.

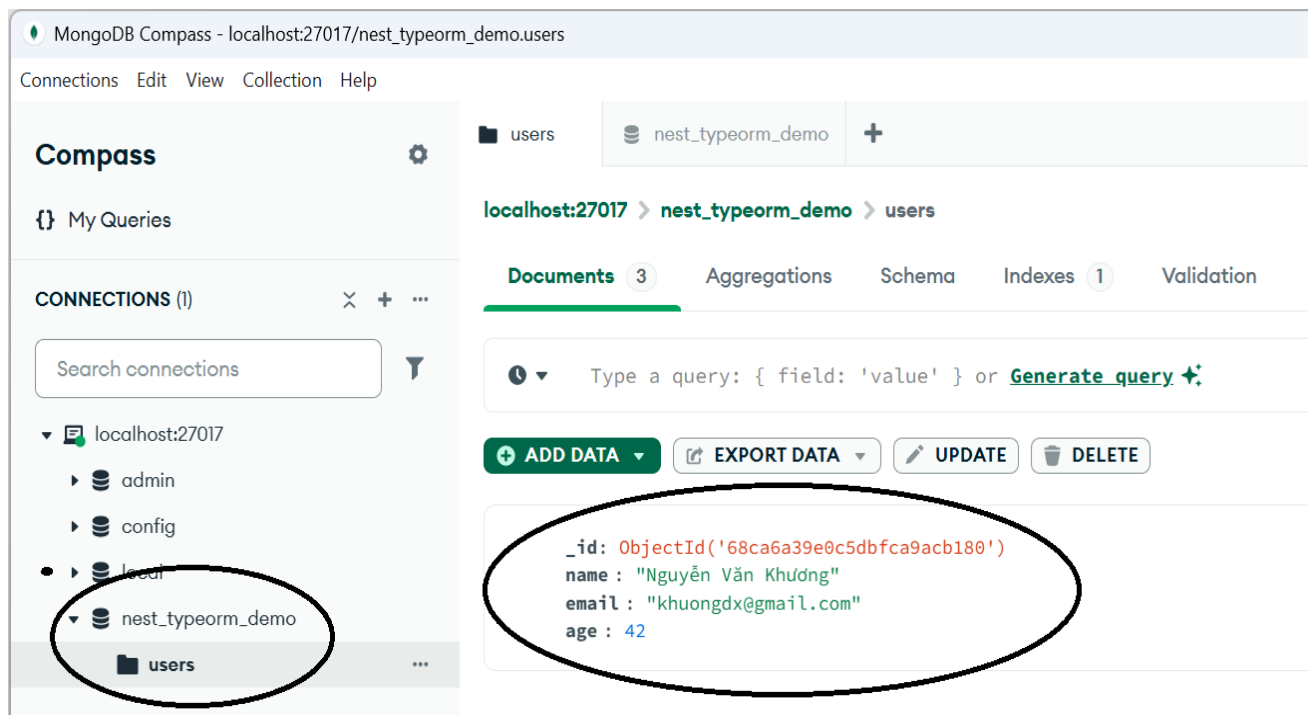
- Bây giờ các bạn mở **Terminal** và biên dịch chương trình: **npm run start:dev**



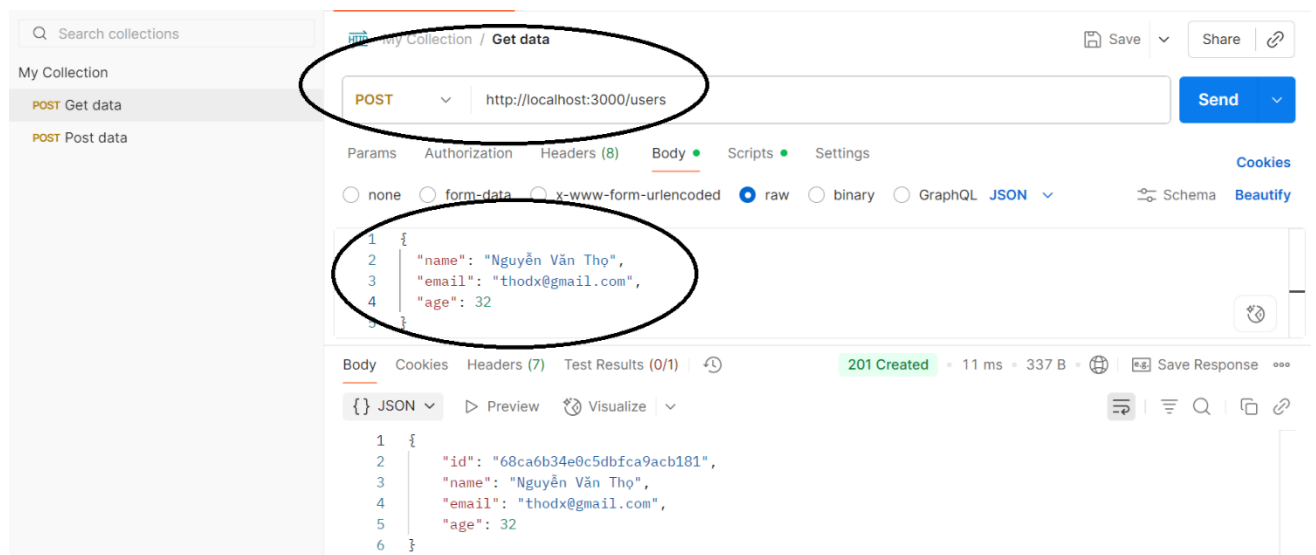
- Để kiểm tra chương trình chạy thế nào các bạn mở chương trình **Postman** lên, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/users**. Chọn thẻ **Body**, chọn **Raw**, gõ nội dung dữ liệu cần thêm vào và chọn **Send**, sẽ thấy kết quả bên dưới:



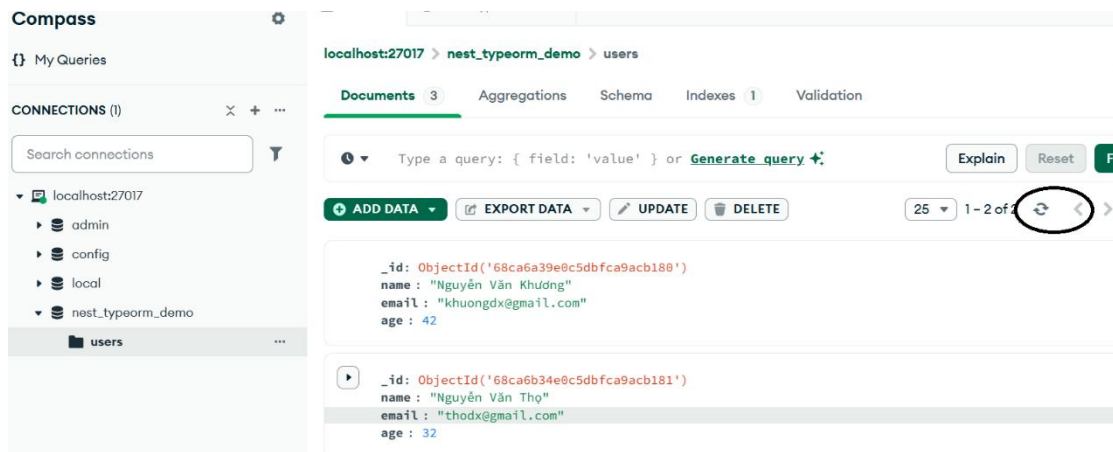
- Mở lại cơ sở dữ liệu sẽ thấy dữ liệu được thêm vào bảng **users** trong Cơ sở dữ liệu



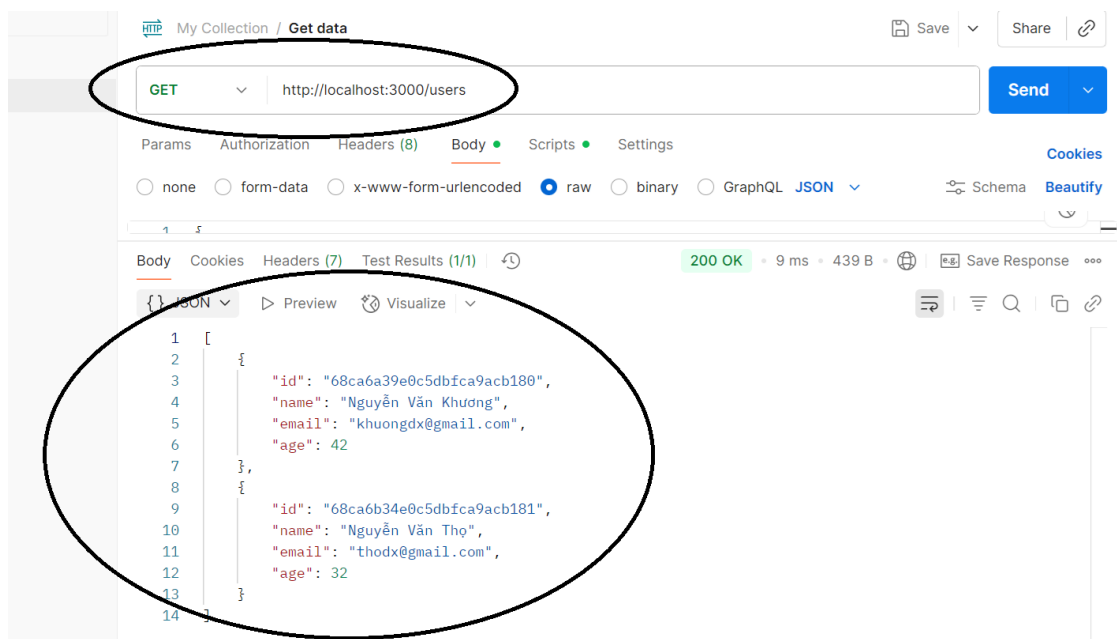
- Tiếp tục ở **Postman** các bạn tạo thêm 1 bản ghi dữ liệu nữa như sau:



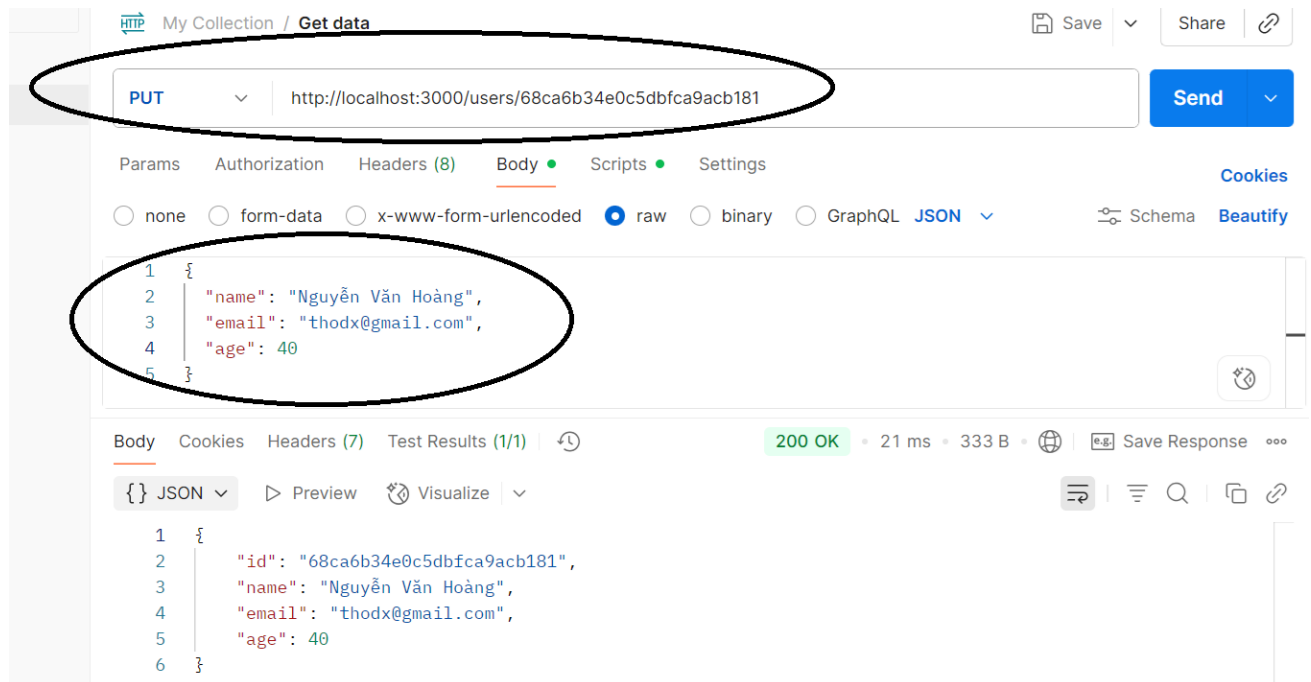
- Các bạn xem lại trong cơ sở dữ liệu **MongoDB** đã có 2 bản ghi dữ liệu được tạo ra:



- Chúng ta tiếp tục thực hiện hàm hiển thị tất cả dữ liệu bằng cách thiết lập trong **Postman** như sau:



- Chúng ta tiếp tục thực hiện hàm cập nhật dữ liệu bằng cách thiết lập trong **Postman** như sau: (id người dùng cần cập nhật các bạn copy trong cơ sở dữ liệu **MongoDB**)



- Chúng ta tiếp tục thực hiện hàm xóa dữ liệu bằng cách thiết lập trong **Postman** như sau: (id người dùng cần cập nhật các bạn copy trong cơ sở dữ liệu **MongoDB**)

